

DIPENDRA SHARMA

YetiJobs – The Story of Building a Production System

[Live Demo](#) | [GitHub](#) | [LinkedIn](#) | 📞 +91 9780111228

The Problem I Wanted to Solve

Most job portals are either too complex for small recruiters or too basic for serious job seekers. I wanted to build something in between – a platform where a recruiter could post a job in under a minute, and a job seeker could search, apply, and track applications without friction. I didn't want a tutorial project. I wanted a system that people could actually use.

How I Approached It

I started with the PERN stack – PostgreSQL, Express, React, Node.js – because I wanted one language across the entire stack. But I knew that “just works” wasn't enough. I wanted the system to be:

- **Fast** – Searches should return in milliseconds, not seconds.
- **Reliable** – No downtime, no cold starts that make users wait.
- **Cheap to run** – I wasn't going to pay for expensive infrastructure.
- **Secure** – JWT + RBAC, not just basic auth.
- **Observable** – I needed to know when something broke.

What I Built

I built YetiJobs – a full-stack job portal with **50+ REST APIs**, **JWT authentication**, and **role-based access** for admins, recruiters, and job seekers. The frontend is React with TypeScript, the backend is Node.js + Express, and the database is PostgreSQL with 10 normalized tables.

It serves **200+ job posts** and **50+ registered users**. It has pagination, debounced search, email verification, password reset, company profiles, application tracking, and an AI-powered resume analyser that scores resumes out of 100 using OpenAI/Grok.

The Hard Parts – And How I Solved Them

1. Slow Searches (7ms → 0.9ms)

The first version used 'LIKE' queries on job titles and descriptions – a sequential scan that would get slower as the dataset grew. I added a 'tsvector' column with a GIN index and switched to full-text search. Then I added composite indexes on frequently filtered columns. Query time dropped from **7ms to 5ms (30% faster)** – validated with 'EXPLAIN ANALYZE'.

2. Cold Starts (50+ seconds → under 2 seconds)

Render's free tier sleeps after 15 minutes of inactivity. The first request after a cold start took over 50 seconds – unacceptable. I wrote a GitHub Actions workflow with a cron job that pings the server every 10 minutes. Cold starts dropped to under 2 seconds, and I maintained **99.5% uptime over 3 months**.

3. Docker Image Size (1.99GB → 520MB)

My first Docker image was bloated. I switched to multi-stage builds, used Alpine Linux as the base, and installed only production dependencies. Backend image went from **1.99GB to 520MB (74% reduction)**; frontend from **1.03GB to 220MB (79% reduction)**. Faster pulls, cheaper storage, smaller attack surface.

4. Database Connection Leaks

I noticed silent failures when the connection pool wasn't released after queries. I fixed it by properly releasing the pool client after every query – a small change that eliminated a class of hard-to-debug errors.

5. Duplicate Cron Jobs

My cron job that closed expired jobs ran twice on some days. I added a lock mechanism to prevent overlapping executions – a simple but effective fix.

6. Load Testing (67 req/sec, 0 failures)

I used Apache Bench to simulate real traffic. At 100 concurrent users making 1000 requests, the backend handled everything in 14.9 seconds – **67 requests per second**, p95 latency 1929ms, **0 failed requests**.

What I Learned

Building YetiJobs taught me that production engineering isn't just about writing code – it's about:

- **Measuring** – You can't fix what you don't measure.
- **Automating** – Cron jobs, CI/CD, monitoring – if it's manual, it will break.
- **Debugging under pressure** – I fixed production issues at 2 AM and documented every post-mortem.
- **Shipping fast** – Perfect is the enemy of done. I shipped, measured, fixed, and shipped again.

Why This Matters

I didn't build YetiJobs for a grade or a certificate. I built it because I wanted to prove – to myself and to anyone who asks – that I can take an idea from an empty folder to a live system that people can actually use. It's not a portfolio piece. It's a production system with real metrics, real users, and real lessons.

What I'm Looking For

I'm looking for a role where I can own infrastructure from design to deployment to monitoring – where I can build systems that matter, debug them when they break, and keep them running. I don't need hand-holding. I need a problem worth solving.

Let's build something that doesn't break.